

Lecture 2: Input Representation

41.601
Agentic AI

etienne@cmkl.ac.th

Content

- Transformers
- Tokenizer
- Embeddings
- Lab



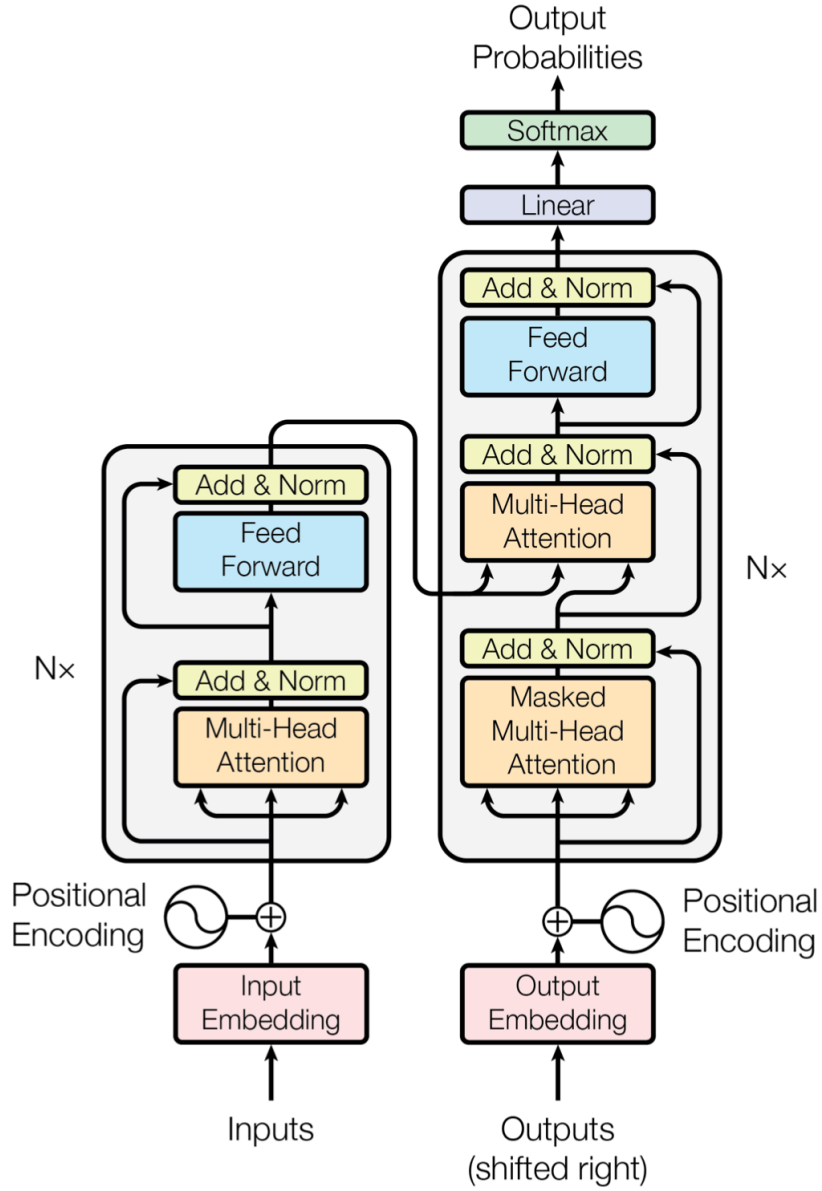
Input Representation

- Neural networks only understand tensors of floating-point numbers
- Text is discrete, variable-length, and culturally specific
- We need a lossless, reversible, learnable mapping: text <-> numbers
- Two separate problems: segmentation (tokenizer) and representation (embedding)



Transfomers

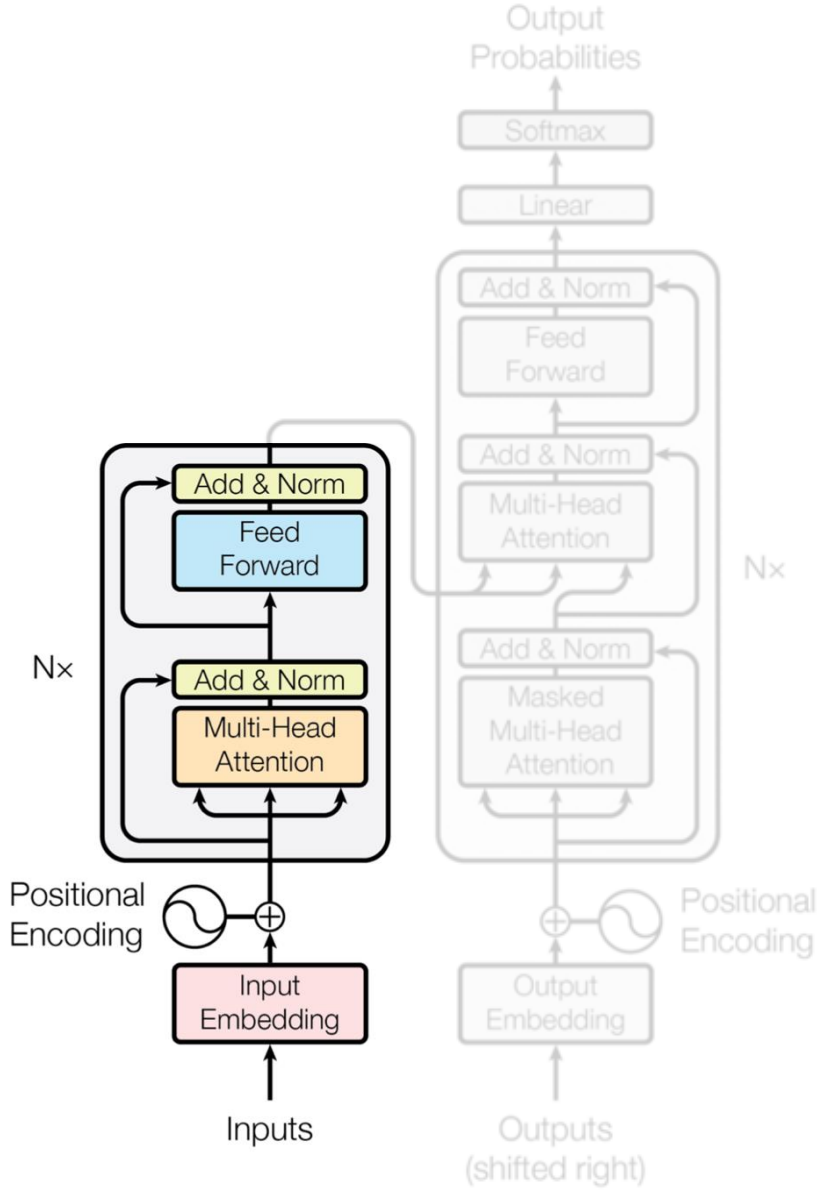
- RNNs: Process tokens one by one, left to right
- Hidden state carries information forward
- Problems: vanishing gradients, no parallelism, forgetting over long sequences
- Transformers: process all tokens simultaneously



Vaswani, Ashish, et al. "Attention is all you need." Advances in neural information processing systems 30 (2017).

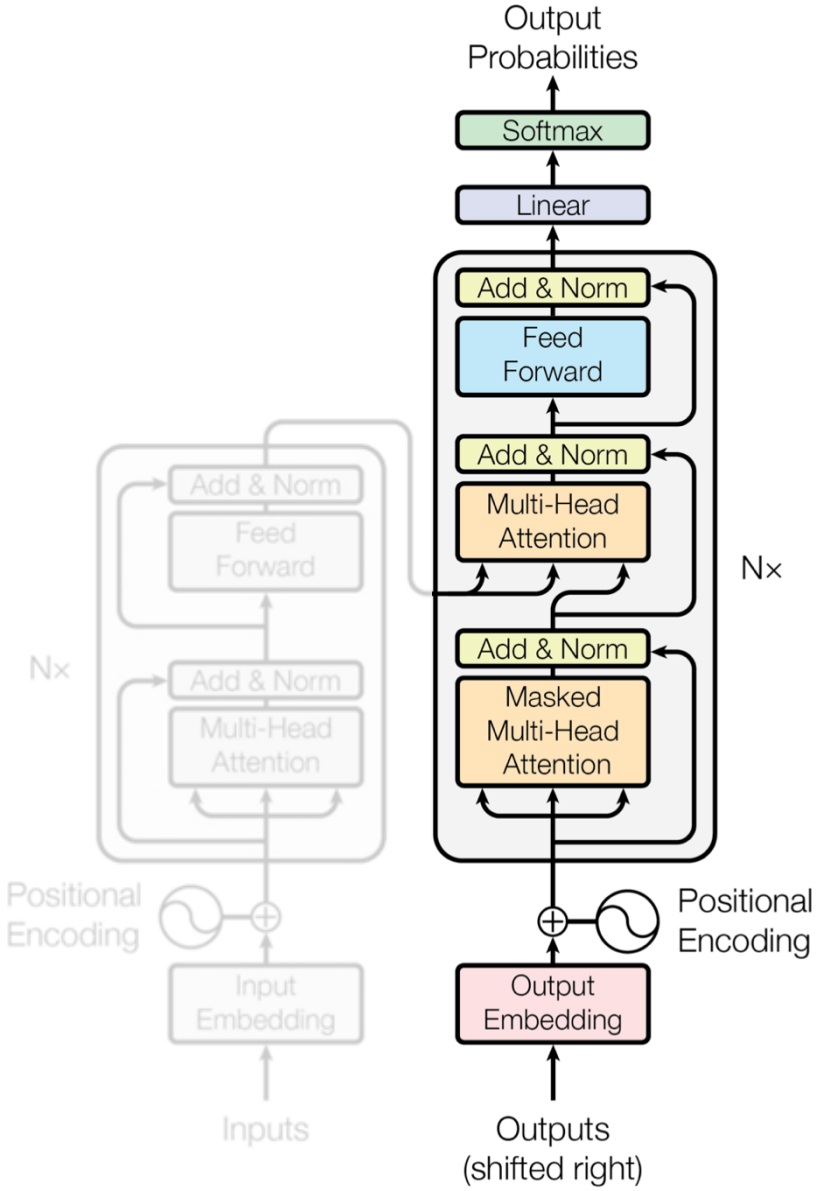
Encoder

- **Encoder-only (e.g. BERT)**
- read the entire input at once, bidirectionally
- Every token can attend to every other token
- Great for *understanding and representing* text:
  - Classification
  - Named entity recognition (NER)
  - Semantic similarity
  - Extracting embeddings
- Cannot generate text



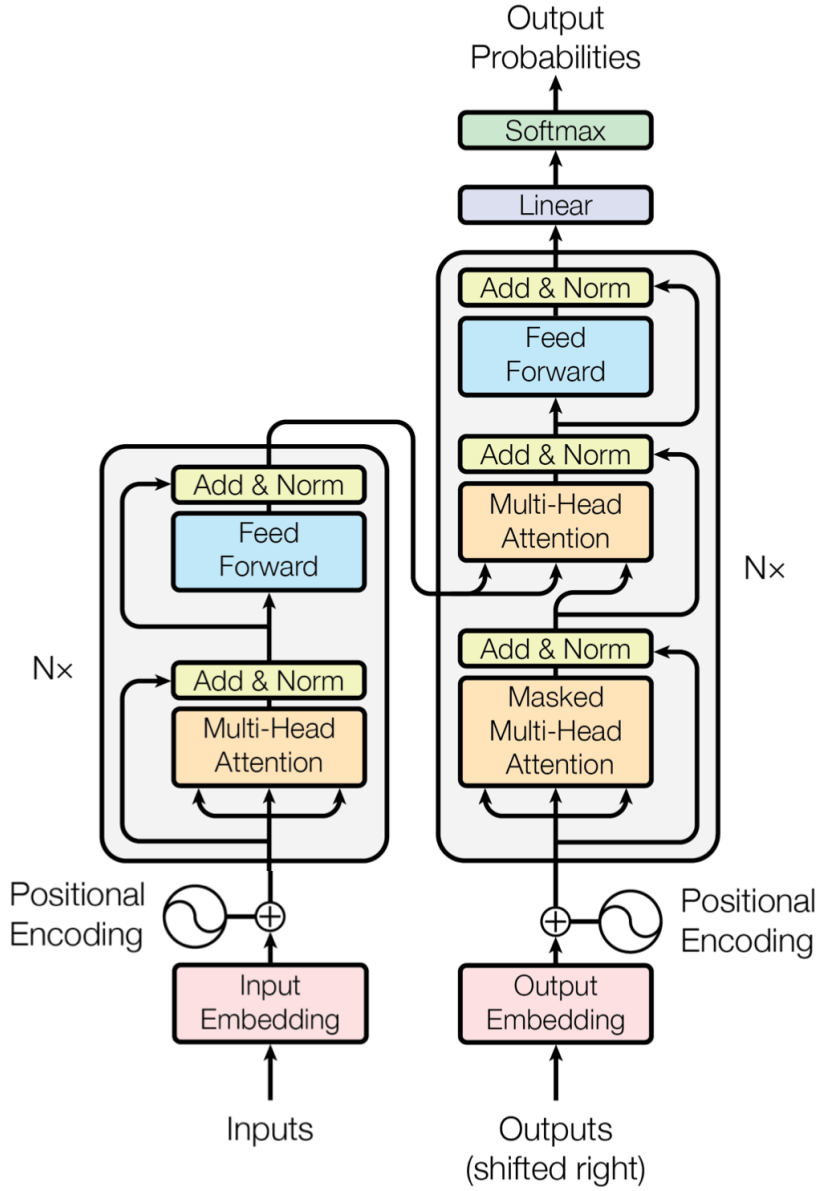
Decoder

- **Decoder-only (e.g. GPT family, LLaMA, Claude)**
- processes tokens left-to-right only (each token can only attend to what came before it)
- Best for *text generation*
- It predicts the next token, one at a time
- State-of-the-art frontier LLMs are decoder-only



Encoder-decoder

- **Encoder-decoder (e.g. T5, BART)**
- use an encoder to build a rich representation of the input, then a decoder to generate an output from it
- *Sequence-to-sequence tasks*
  - Translation
  - Summarization
  - Question answering
- Encoder "understands", decoder ”produces"



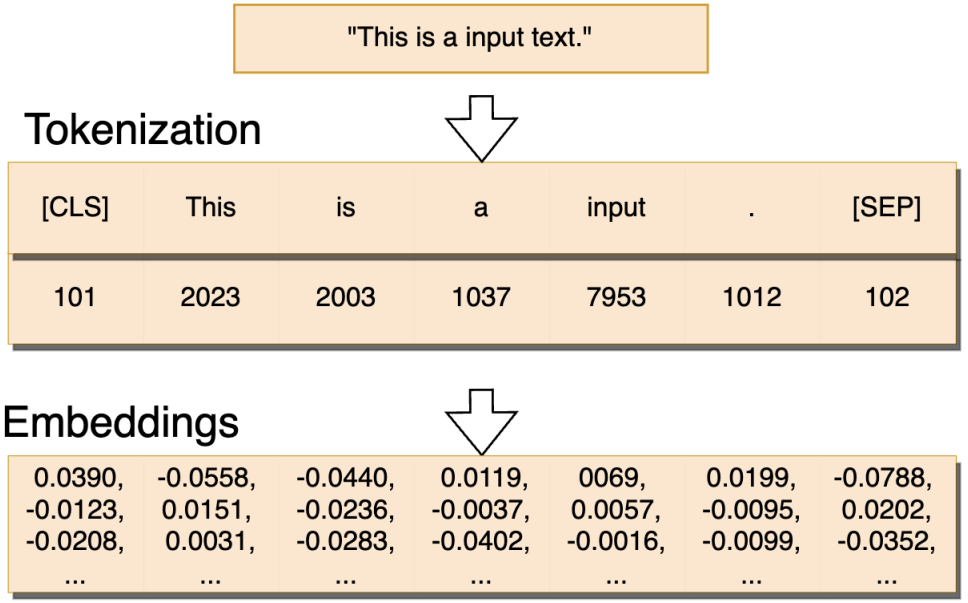
A Brief History of Text Representations

- One-hot encoding: simple, but vocabulary-sized and no semantics
- TF-IDF: sparse, keyword-based, still used in search
- Word2Vec (2013): dense, semantic, but static
- ELMo (2018): contextual but RNN-based
- Today: Transformer embeddings: contextual + parallelizable



The Full Pipeline

1. Raw string
2. Tokenizer
3. Token IDs
4. Embedding lookup
5. Contextual vectors
6. LLM



- The tokenizer is not learned during LLM training (it's fixed beforehand)
- The embedding table is learned during training



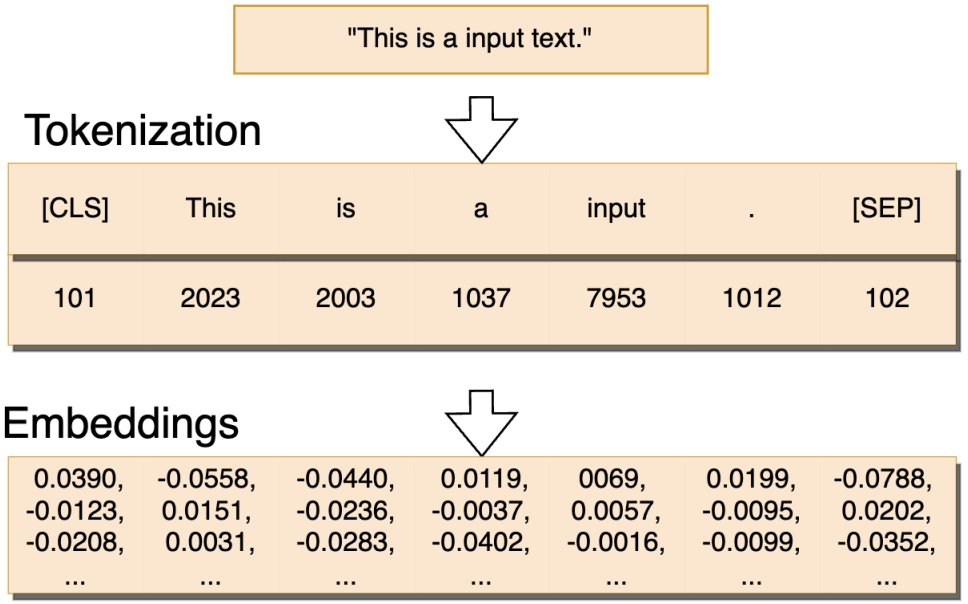
Content

- Transformers
- Tokenizer
- Embeddings
- Lab



Tokenizer

A tokenizer converts raw text into a sequence of **token IDs** by splitting text into subword units



What Is a Token?

- A token is the atomic unit the model processes (not necessarily a word)
- Examples: "unbelievable" → ["un", "believ", "able"] or ["_un", "believable"]
- Rule of thumb: ~¾ of a word on average in English
- Why not just characters?
  - ➢Too long sequences, loses word-level structure
- Why not just words?
  - ➢Unseen words, huge vocabulary

- See: https://platform.openai.com/tokenizer



Vocabulary and Token IDs

- The vocabulary is a fixed mapping: string fragment → integer
- Example: GPT-4 uses ~100,277 tokens; BERT uses 30,522
- Special tokens: `[CLS]`, `[SEP]`, `<|endoftext|>`, `<|im_start|>`
- The model never sees strings again after this step



Training a Tokenizer

- Text corpus + target vocabulary size
- The `tokenizers` library (HuggingFace) makes this ~20 lines of code
- Why train your own?
  - Domain-specific corpora (medical, legal, code), new languages, efficiency optimization
- Trade-off: custom tokenizer means you can't reuse pre-trained model weights directly



Example: Byte-Pair Encoding (BPE)

```
> Initialize vocab with all characters + special tokens
> Count all adjacent pairs in the corpus
> Merge the most frequent pair; add it to vocab
> Repeat N times (N = target vocab size – initial size)
> Finished when vocabulary reaches target size
> Result: common words become tokens, rare words split to subwords

> The trained merge table is the tokenizer
```

https://mbrenndoerfer.com/writing/byte-pair-encoding-subword-tokenization-guide



Tokenizers Overview

- **BPE (GPT):** iteratively merges the most frequent adjacent character pair in the corpus until the target vocabulary size is reached
- **WordPiece (BERT):** maximizes language model likelihood instead of raw frequency
- **SentencePiece (LLaMA):** treats the raw byte stream, not pre-tokenized words; language-agnostic, handles spaces as explicit characters (_)



Issue 1: Numbers & Arithmetic

- "42" might be one token; "43" might be two; "1000000" might be six
- The model has no built-in notion of numerical magnitude

➢LLMs struggle with arithmetic without tools



Issue 2: Multilingual Inequality

- "Hello" = 1 token; "Héllo" = 3 tokens; Thai/Arabic sentences = 2–5× more tokens than English equivalents
- Consequence: non-English users pay more, get shorter effective context
- Research area: training balanced multilingual vocabularies
- Show a table: same sentence in 5 languages, token counts



Issue 3: Whitespace, Case & Punctuation

- Leading spaces matter: “ London” ≠ “London” in GPT tokenizers
- Case sensitivity varies by tokenizer: “Hello” ≠ “hello” ≠ “HELLO”
- Punctuation often merges with adjacent words: “World” + “!” -> “World!”
- Why this matters: prompt engineering is implicitly token engineering



Context Window

- Context window = max token sequence the model can process at once
- Longer documents = more tokens = may exceed the window
- Cost (API pricing) is per token, not per character
- Try estimating token count before making API calls



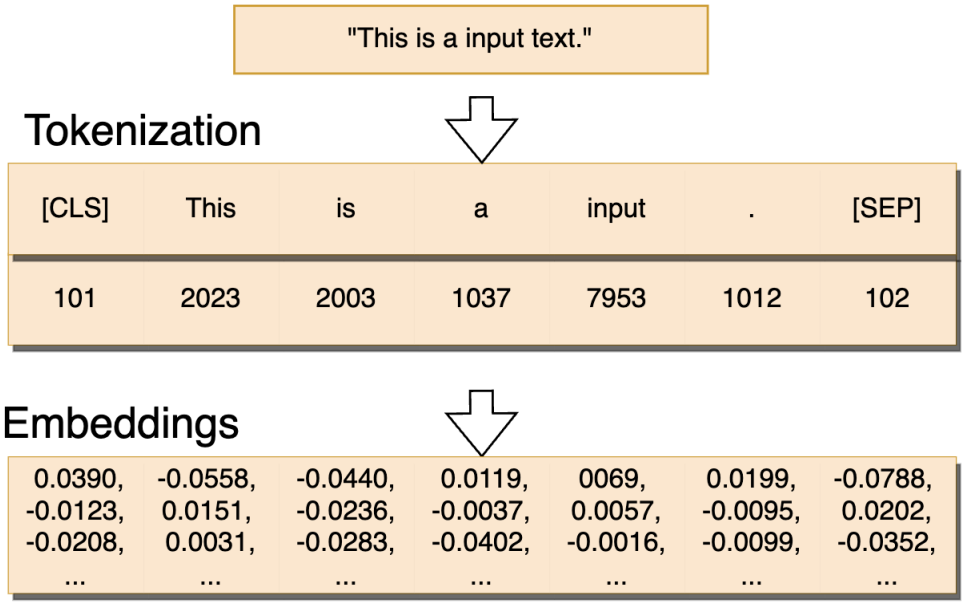
Content

- Transformers
- Tokenizer
- Embeddings
- Lab



Embeddings

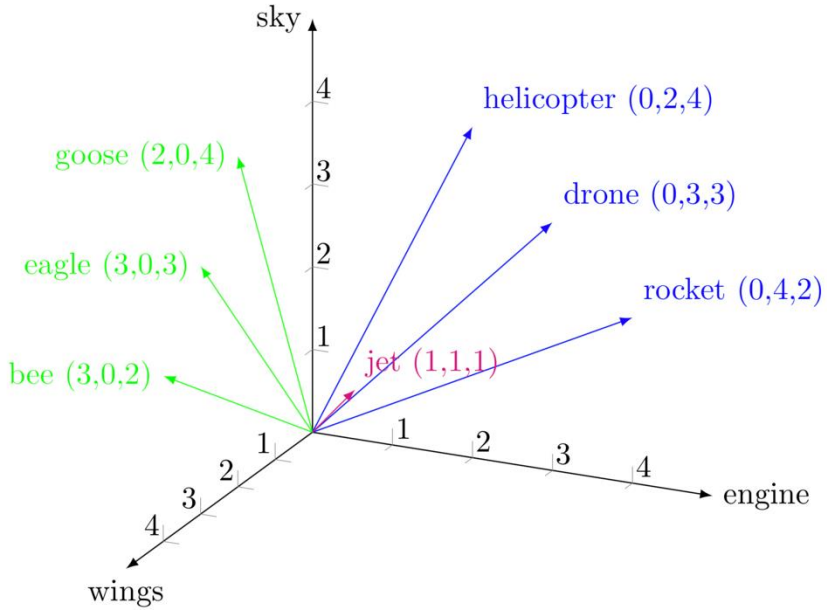
An embedding turns a token ID into a vector that represents its meaning (similar tokens point in similar directions)





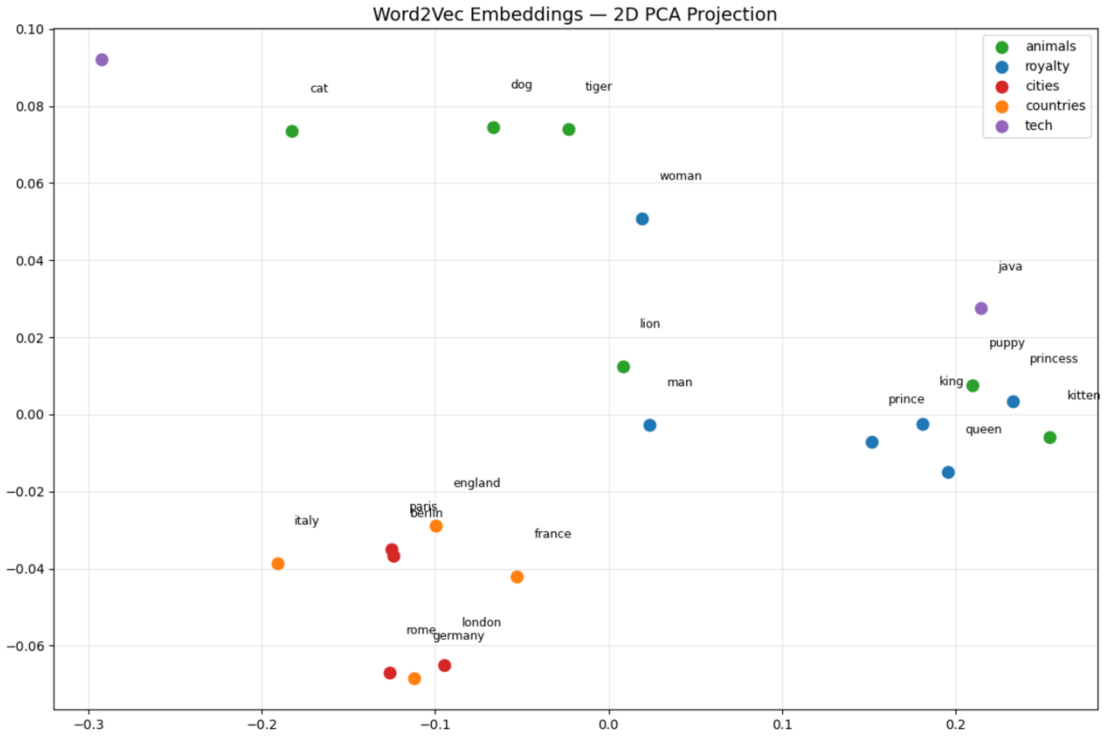
Training the Embedding Table

- The embedding table is initialized randomly and updated continuously during LLM training
- Similar contexts are getting closer in vector space
- The resulting matrix encodes meaning: direction represents semantics, similar words point in similar directions
- Embedding table shape: [vocab_size × d_model]
- Row lookup: token ID 9906 → d-dimensional vector
  - OpenAI: `text-embedding-3-small` (1536d), `text-embedding-3-large` (3072d)
  - Open-source: `all-MiniLM-L6-v2` (384d), `bge-m3` (multilingual), `nomic-embed-text` (768d)



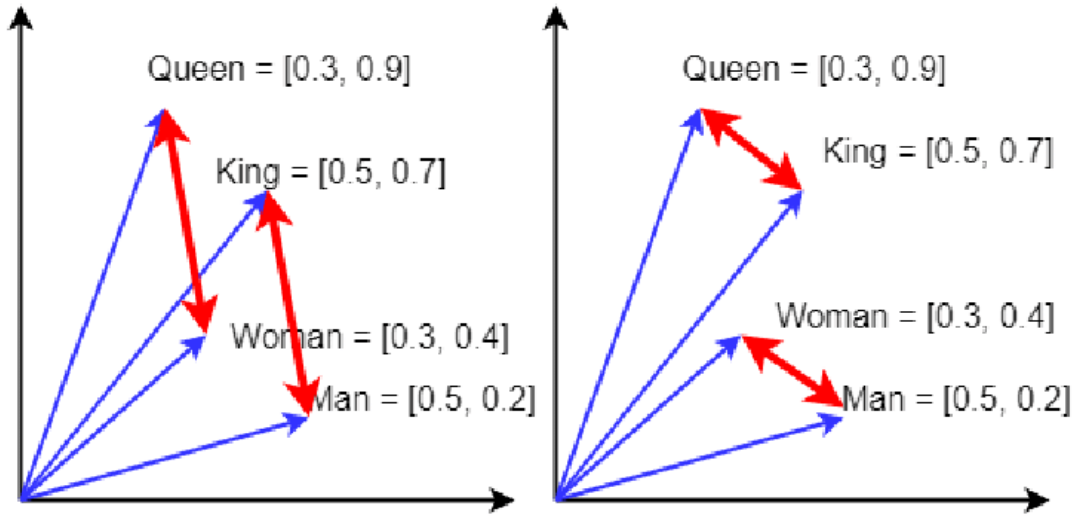
The Geometry of Meaning

- Vectors in high-dimensional space cluster by semantic similarity
- Cosine similarity as a measure of "direction" rather than magnitude
- Key property: semantic relationships become geometric relationships
- Visualization: 2D PCA projection of word vectors (-> Lab)



Lab: Word2Vec

- Skip-gram: given a word, predict surrounding words
- CBOW: given surrounding words, predict the center word
- Training signal is self-supervised: no labels needed
- The famous analogy:
  "king" - "man" + "woman" ≈ "queen"



Word2Vec: What It Learns & What It Doesn't

- Syntactic relationships: plural, tense, comparative
- Semantic relationships: capital cities, professions, categories
- Fatal limitation: one vector per word regardless of context
- "The river bank" vs "the savings bank" → same vector



Contextual Embeddings in Transformer

- Self-Attention: Every token gets a different vector depending on context
- "bank" in two sentences → two different vectors, potentially far apart
- Key innovation that made LLMs so powerful!



Positional Embeddings

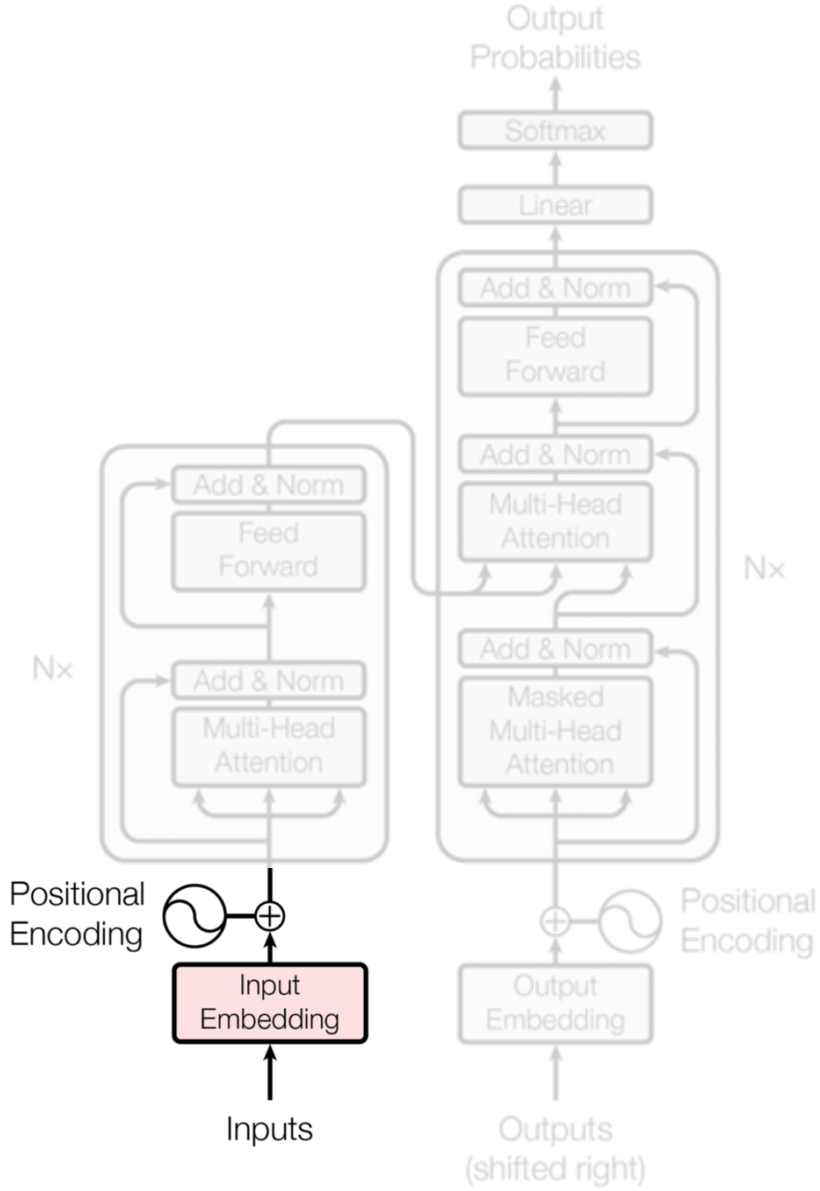
"This is a input text."

Tokenization

[CLS]	This	is	a	input	.	[SEP]
101	2023	2003	1037	7953	1012	102

Embeddings

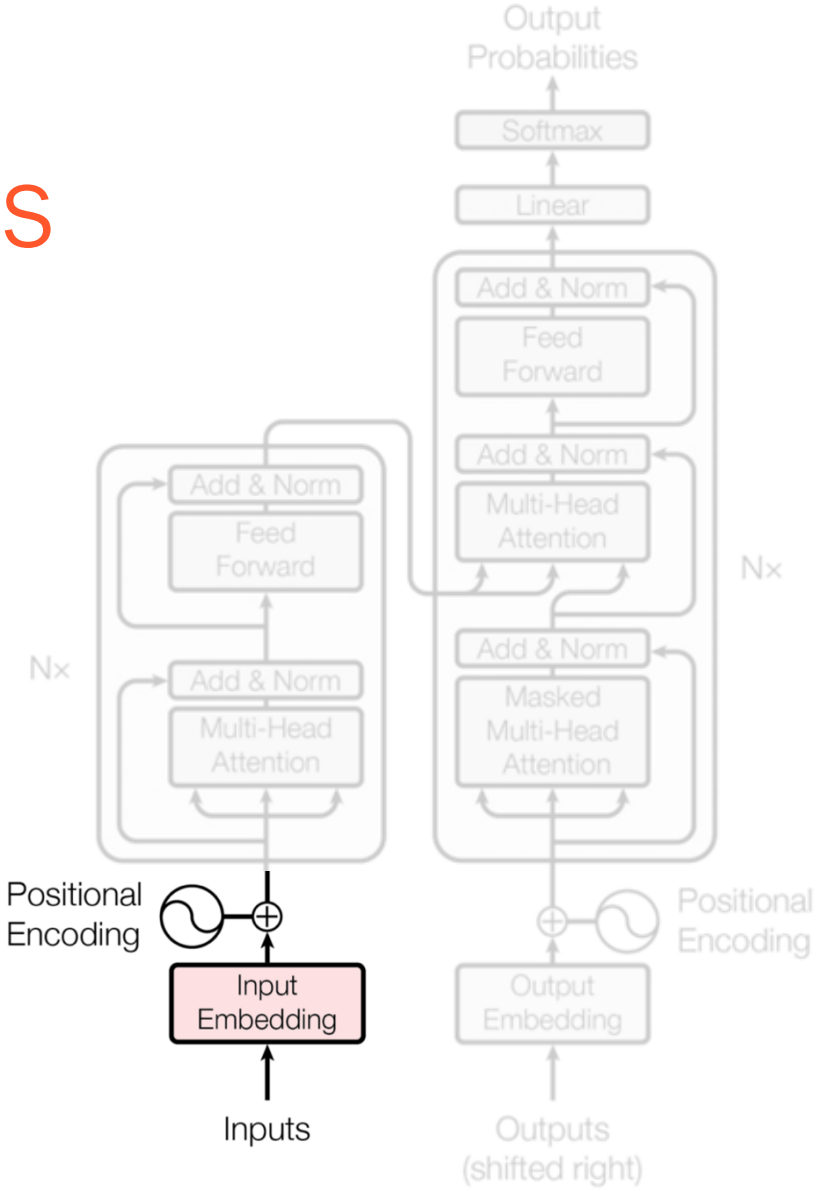
0.0390, -0.0123, -0.0208, ...	-0.0558, 0.0151, 0.0031, ...	-0.0440, -0.0236, -0.0283, ...	0.0119, -0.0037, -0.0402, ...	0069, 0.0057, -0.0016, ...	0.0199, -0.0095, -0.0099, ...	-0.0788, 0.0202, -0.0352, ...



- Tokenizer
- Embedding vector `[vocab_size × d_model]`
- + Position encoded as vector, added to Embeddings `token_embed[i] + pos_embed[i]`
- Two types: absolute and relative positional encoding

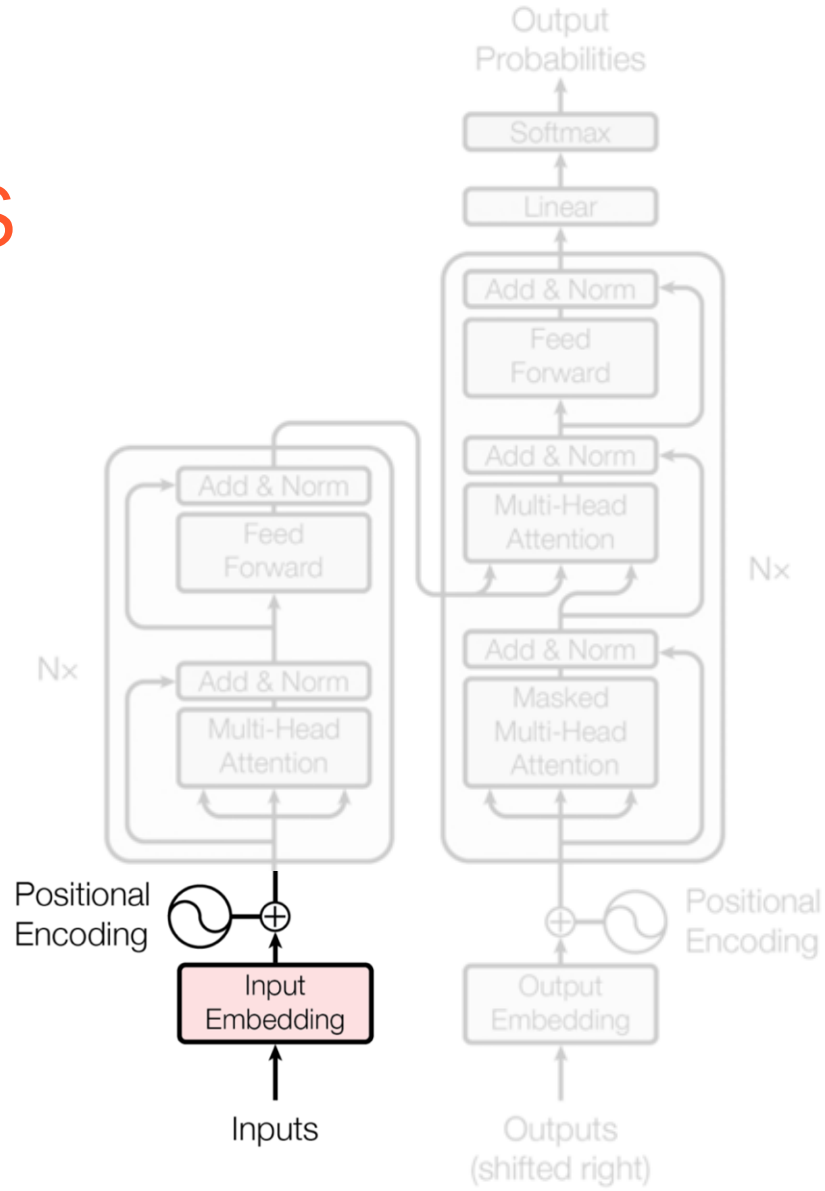
Absolute Positional Embeddings

- Used in original Transformer
- Learned: a trainable vector for each position 0…T
- Vector added to token embedding
- Sinusoidal (original Transformer): fixed sine/cosine waves of different frequencies
- Limitation: position is encoded in the embedding, not in attention -> hard to extrapolate beyond the training context length



Relative Positional Embeddings

- RoPE (Rotary Position Embedding) (LLaMA, GPT-4)
  - Rotates Q and K by an angle proportional to their absolute position
  - Q·K captures relative distance between tokens
  - Better length extrapolation; computationally efficient
- ALiBi (Attention with Linear Biases)
  - Subtracts a linear penalty `m × |i − j|` from attention scores before softmax
  - No positional vectors at all — bias is fixed, not learned
  - Strong out-of-the-box extrapolation to longer sequences
- RoPE has become the dominant choice in modern open-weight LLMs



Content

- Transformers
- Tokenizer
- Embeddings
- Lab



Lab

- Exploring Tokenization
- Training a BPE Tokenizer
- Training Word2Vec Embeddings



ขอบคุณครับ

Thank you!

etienne@cmkl.ac.th

